

Документация к GPU Sandbox

Содержание

1	Назначение программы	2
2	Архитектура	2
2.1	Memory	2
2.2	Core	2
2.2.1	Регистры	2
2.2.2	Программный счётчик	2
2.2.3	Текущее состояние	2
2.3	Instructions	3
2.3.1	Операнды	3
2.3.2	Opcode	3

1 Назначение программы

GPU Sandbox представляет собой программную модель вычислительного процессора.

2 Архитектура

На текущем этапе модель содержит одно вычислительное ядро, память и программу, состоящую из последовательности инструкций.

2.1 Memory

`data[MEMORY_SIZE]` — это память виртуального GPU. Именно в ней хранятся все данные, с которыми выполняются операции.

Количество элементов памяти определяется переменной `MEMORY_SIZE`. Изначально её значение равно 1 МБ. В будущем это значение будет изменено.

2.2 Core

Core — ядро процессора. В нём находятся следующие компоненты.

2.2.1 Регистры

`registers[REGISTERS_COUNT]` — это массив регистров, которые хранят в себе промежуточную информацию, с которой впоследствии будут произведены операции.

Количество регистров определяется переменной `REGISTERS_COUNT`, которая изначально равна 8. В будущем это значение будет изменено.

2.2.2 Программный счётчик

pc — программный счётчик (от англ. *Program Counter*). Это специальный компонент, который указывает на инструкцию (команду), которую видеокарта должна выполнить следующей.

Максимальное значение счётчика команд измеряется длиной текущей инструкции.

2.2.3 Текущее состояние

`current_state` — это компонент, отвечающий за текущее состояние ядра.

Состояния бывают следующие:

1. **NORMAL** — нормальное состояние ядра. Это состояние, которое принимает ядро во время работы при условии, что всё работает в штатном режиме без ошибок.
2. **JUMP** — состояние, сигнализирующее о том, что ядро перепрыгивает с инструкции, которую оно должно было исполнить следующей, на инструкцию с индексом n .

Подробнее данная операция рассматривается ниже.

3. **HALT** — состояние, сигнализирующее о прекращении выполнения программы ядром.

4. **ERROR** — состояние, которое означает, что ядро столкнулось с какой-либо ошибкой, из-за чего экстренно завершило свой процесс.

Обычно появляется при неправильно заданной инструкции.

5. **TIMEOUT** — состояние, схожее с **ERROR**, но отличается тем, что программа превысила максимально возможное количество циклов.

Один цикл равен одной инструкции. Максимальное количество циклов до перехода ядра в состояние **TIMEOUT** определяется переменной **MAX_CYCLES**.

Изначально её значение равно 1 000 000. В будущем это значение может быть изменено.

2.3 Instructions

Instructions — это инструкции (команды), которые должно выполнить ядро.

Для грамотной работы с инструкциями также нужны следующие компоненты.

2.3.1 Операнды

a, **b**, **c** — операнды, в которых записывается информация, которую будет использовать ядро для выполнения текущей операции. Данные наименования операндов в будущем могут быть изменены

2.3.2 Opcode

Opcode (образовано от *Operation Code*) — это номер операции, который определяет, какую операцию нужно выполнить. Наглядная визуализация распределения операндов для работы опкода находится в таблице 1.

На данный момент в программе присутствуют следующие опкоды:

1. **OP_LOAD** — операция, которая позволяет считать из памяти значение по адресу, хранящемуся в регистре **b**, и записать его в регистр **a**.
2. **OP_STORE** — операция, позволяющая записать значение регистра **b** в память по адресу, хранящемуся в регистре **a**.
3. **OP_JUMP** — операция, позволяющая перейти на индекс инструкции, сохранённый в регистре **a**.
4. **OP_ADD** — операция, которая записывает в регистр **a** сумму значений из регистров **b** и **c**.
5. **OP_SUB** — операция, которая вычитает значение регистра **c** из регистра **b** и записывает результат в регистр **a**.
6. **OP_HALT** — операция, означающая конец выполнения программы. Она переводит ядро в состояние **HALT**.

Таблица 1: Набор инструкций виртуального процессора

Opcode	a	b	c	Назначение
OP_LOAD	регистр	регистр с адресом	—	Загрузка значения по адресу из b в a .
OP_STORE	регистр с адресом	регистр со значением	—	Запись значения по адресу из a .
OP_JUMP	регистр с индексом инструкции	—	—	Переход к инструкции, адрес которой хранится в регистре.
OP_ADD	регистр	регистр	регистр	Сложение значения в c с записью регистра a в b .
OP_SUB	регистр	регистр	регистр	Вычитание значения в c из регистра a с записью результата в b .
OP_HALT	—	—	—	Прекращение выполнения программы и переход к состоянию HALT.